cyber673

HACK101: Mobile App

Introduction to Mobile Application Vulnerability Assessments & Penetration Testing

cyber673.com

About Me

Izdihar Sulaiman

Cybersecurity Professional

- Founder of cyber673 community
- Penetration tester & security assessor
- HACK101 workshop series organiser

Disclaimer

Educational Purposes Only

Everything taught in this workshop is strictly for educational and authorised testing purposes. Unauthorised testing of any systems, networks, or websites is illegal.

Computer Misuse Act (Brunei)

- Criminalises unauthorised access to computer systems
- Penalties include fines and imprisonment
- Always obtain written authorisation before testing
- Applies even to "white-hat" or well-intentioned testing

What is HACK101?

A series of introductory workshops covering various types of technical security assessments (VAPTs)

Network *Completed*	**Web App** *Coming soon*	**Mobile** *Today!*
Active Directory *Coming soon*	**Thick Client** *Coming soon*	**IoT** *Coming soon*

Pentesting Lifecycle

1
Recon
Gather information about the target

2
Vuln Assess
Identify potential vulnerabilities

3
Exploit
Attempt to exploit findings

4
Report
Document findings

5
Cleanup
Remove traces and restore

The Technical Flow

The VAPT Business Process

01 **Get Project**

Client engagement, scoping discussions, proposal and SOW

02 **Planning & Scope**

Confirm scope, APK, rules of engagement, testing windows

03 **Initial Test**

Perform the VAPT, document findings as you go

04 **Initial Report**

Deliver findings report with risk ratings and recommendations

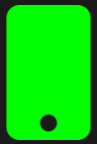
05 **Retest**

Client remediates; verify fixes in a retest cycle

06 **Retest Report**

Final report confirming remediation status of each finding

Mobile App VAPTs



What is a Mobile App VAPT?

A systematic assessment of mobile applications (Android/iOS) to identify security vulnerabilities in code, architecture, data storage, and communication that could compromise user data or device integrity.

Key Areas

- Static analysis: decompile APK, review code for hardcoded secrets
- Dynamic analysis: runtime behavior, Burp Suite interception
- Data storage: SharedPreferences, SQLite, external storage
- Communication: HTTPS/TLS, certificate pinning, API security

Setting Up Your Lab

Tools and environment setup for mobile app pentesting

Android Emulator	Android Studio or GENYMOTION - create rooted virtual device
ADB (Android Debug Bridge)	$ adb devices — connect to emulator, install APK, access shell
Frida	$ frida-server on device, hook functions at runtime, bypass root detection
APKTool / JADX	Decompile APK to source code and resources
Burp Suite / Mitmproxy	Setup proxy on device to intercept HTTPS traffic, install CA cert

Static Analysis

Analyzing APK without Running It

```
$ jadx app.apk -d output/  (or apktool d app.apk)
```

Key Areas to Review

- AndroidManifest.xml — permissions, activities, intent filters, debuggable flag
- Source code (decompiled) — hardcoded secrets, weak crypto, insecure APIs
- strings.xml / resources — leaked API keys, Firebase URLs, backend URLs
- gradle/build configs — dependencies, build variants, debug keys

Dynamic Analysis

Analyzing APK at Runtime

Burp Suite Setup

- Configure Android emulator proxy to route traffic through Burp
- Install Burp's CA certificate in the device's trusted store
- Intercept HTTP/HTTPS traffic to inspect requests and responses
- Modify requests on-the-fly to bypass authentication or test injection

Frida Hooking

- Inject custom JavaScript into app at runtime to bypass security checks

Finding: Insecure Data Storage

The Issue

Sensitive data stored unencrypted or weakly protected on device.

Examples:

- Passwords in SharedPreferences (plaintext)
- SQLite database with unencrypted PII
- Log files containing credentials
- Cache files with sensitive info

Detection

```
$ adb shell
```
Browse /data/data/[package]/ directories

```
$ adb pull /data/data/.../
```
Extract SharedPrefs and databases

```
$ sqlite3 app.db
```
Query database contents

Risk Rating:
High (PII/Credentials exposed)

Finding: Insecure Communication

HTTP Traffic or No Certificate Pinning

App communicates over plain HTTP or accepts any HTTPS cert without pinning.
Attacker can MITM traffic, intercept credentials, modify API responses.

Detection (Static)

- Search for "http://" in source code
- Check for certificate pinning logic
- Review SSL/TLS validation code

Look for trust-all trust managers

Detection (Dynamic)

- Use Burp Suite / Mitmproxy proxy
- Intercept and inspect traffic
- Attempt to modify requests

Check if HTTPS validates certs

Finding: Hardcoded Secrets

The Issue

API keys, credentials, tokens embedded in app code or resources.

Examples:

- AWS/Azure API keys in strings.xml
- Firebase API keys and URLs
- Backend service credentials
- JWT tokens, OAuth tokens

Detection & Exploitation

```
$ strings app.apk | grep -i api
$ strings app.apk | grep -i firebase
```

Search decompiled code for hardcoded strings

Impact:
Direct API abuse, account takeover, data access

Risk Rating:
Critical

Remediation Guide

Insecure Data Storage

- Use EncryptedSharedPreferences
- Encrypt SQLite with SQLCipher
- Never log sensitive data
- Disable world-readable storage
- Wipe data on uninstall

Insecure Communication

- Enforce HTTPS only (network_security.xml)
- Implement cert pinning
- Validate all certificates
- Use modern TLS 1.2+ only
- Strong ciphers only

Hardcoded Secrets

- Never hardcode credentials
- Use backend API for auth
- Rotate API keys regularly
- Obfuscate code (ProGuard/R8)
- Use secure config management

CVSS 3.1: Severity Ratings

Common Vulnerability Scoring System - the industry standard for rating vulnerability severity

None	Low	Medium	High	Critical
0.0	0.1-3.9	4.0-6.9	7.0-8.9	9.0-10.0

Base Score Metrics

Exploitability:

- Attack Vector (Network/Adjacent/Local/Physical)
- Attack Complexity (Low/High)
- Privileges Required (None/Low/High)
- User Interaction (None/Required)
- Scope (Unchanged/Changed)

Impact Metrics

CIA Triad:

- Confidentiality Impact (None/Low/High)
- Integrity Impact (None/Low/High)
- Availability Impact (None/Low/High)

Calculator:
nvd.nist.gov/vuln-metrics/cvss/v3-calculator

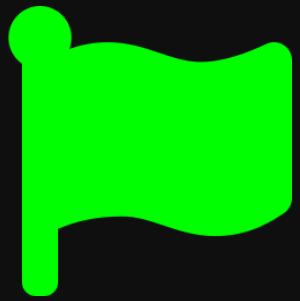
cyber673

LIVE DEMO

Let's put theory into practice

Testing a vulnerable Android app

cyber673



CTF CHALLENGE

8 challenges across 1 APK + 1 backend - find the flags!

hack101.cyber673.com

cyber673

Thanks for hacking with us!

Join the community at cyber673.com

Discord | Instagram | cyber673.com